Types and Programming Languages – Notes

goldenstack

2026-10-16

Contents

1. Introduction 2
   1.1. Types in Computer Science 2
   1.2. What Type Systems Are Good For 2
   1.3. Type Systems and Language Design 3
   1.4. Capsule History 3
   1.5. Related reading 3
2. Mathematical Preliminaries 4
   2.1. Sets, Relations, and Functions 4
   2.2. Ordered Sets 4
   2.3. Sequences 5
   2.4. Induction 5
   2.5. Background Reading 6
3. Untyped Arithmetic Expressions 7
   3.1. Introduction 7
   3.2. Syntax 8
   3.3. Induction on Terms 9
   3.4. Semantic Styles 9
   3.5. Evaluation 10
Select References 12

1. Introduction

1.1. Types in Computer Science

Formal methods vary from powerful but cumbersome (Hoare logic, denotational semantics) to modest yet lightweight (type systems). The textbook focuses more on the latter variation, type systems. Type systems are generally either more practical or more abstract; once again, the textbook focuses on the more pragmatic aspect, practical type systems.

The definition of a 'type system' is rather hard to pin down. An example:

> *A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute.*

Type systems are typically used to *statically* prove information about a program (see Curry-Howard Correspondence). As with any system involving proofs, it is inherent to the system that some true statements cannot be proven. In the case of type systems, this is called being *conservative* and is represented by the type system rejecting an nonzero number of programs that do *not* display the "certain program behaviours".

The conservativity of a type system can generally be reduced by allowing more specific types, typically with a more general type system, so that certain structures and relationships can be represented more accurately. Likewise, a type system can only detect certain classes of errors; thes classes can only be expanded by allowing more generality in the type system.[1] As type systems grow to support more extended static checking methods, they approach being a proof checker.

1.2. What Type Systems Are Good For

Detecting errors: A proof system running automatically on your code allows for many errors to be detected automatically, hastening correctness.

Abstraction: Type systems can represent contracts that must be upheld, allowing more modularity by virtue of relying on contracts (types), not definitions.

Documentation: Types also act as a form of documentation in that they desribe an abstracted form of your code; importantly, your code and its types cannot become desynchronized.

Safety: Although safety is hard to define, a language is often considered 'safe' if it protects its own abstractions well with some sort of static or dynamic type safety. Several definitions of safety:

- Protecting its own abstractions (if the program is unsafe, it may violate its abstractions)
- Preventing untrapped errors at runtime (if the program is unsafe, errors may propagate)
- Completely defined by the manual (i.e. no undefined behaviour)

Efficiency: Type checkers provide more compile-time guarantees, allowing for more optimizations, such as which instructions are used or which runtime checks can be omitted.[2]

Other uses:

Security: Unsafe languages can be leveraged in ways that harm security (see Microsoft on memory safety). Type systems that prevent this can help increase the security of programs.

[1] Generality is not always the best term to describe this behaviour. For example, supporting the access of private data emitting an error requires arguably a more specific type system, to handle an invented feature.

[2] I do take issue with this statement. The point at which one has the most information about program functionality and tendencies is at runtime; thus, there is a higher performance ceiling when concerned with runtime checks (see Itanium as a prominent pro-runtime/pro-JIT example). Furthermore, a sufficiently intelligent compiler can hypothetically determine all information that a type-checker would be able to—it is simply much more difficult to write a compiler as such. In a way, efficiency *does* apply here—just as compiler development efficiency instead of program runtime efficiency. I digress.

Data storage: Types are on a rise in database or data storage systems, with JSON schemas and `.xsd` files as just two examples. They help structure data, and have all the same benefits types do normally.

1.3. Type Systems and Language Design

Type systems, like any other feature, should not be bolted on to an existing language. It is best when all features are designed hand-in-hand. This is exemplified for type systems specifically as they are often core features of a language, around which syntax, semantics, and functionality are defined.

Note that the importance of a type system to any particular language is unrelated to the two separate problems of:

- Having dynamic (Python, etc.) versus static typing (C, etc.).
- Having implicit (ML, etc.) versus explicit type annotations (Java, etc.).

1.4. Capsule History

In the early to mid 1950s, type systems were first used to distinguish between integer types and floating-point types. In the late 1950s and early 1960s, this was extended to include basic structured data (such as arrays of records) and higher order functions. Later, in the 1970s, these were expanded into richer concepts like abstract data types, modules, subtypes, and parametric polymorphism. Around this time, computer scientists began to draw parallels between these type systems and more formal mathematical logic.

The original textbook has an extended timeline. A few select choices[3]:

- 1900s: Formalization of mathematics
- 1930s: Untyped lambda calculus
- 1960s: Curry-Howard Correspondence
- 1970s: Intuitionistic type theory, ML, and ADTs
- 1980s: Linear logic, the calculus of constructions, dependent types, and effect systems.
- 1990s: Typed assembly

The textbook was produced in 2002 and thus has nothing past the 1990s.

1.5. Related reading

- *Foundations for Programming Languages* by Mitchell (1996)
- *Theories of Programming Languages* by Reynolds (1998)
- *The Structure of Typed Programming Languages* by Schimdt (1994)
- *Basic Simple Type Theory* by Hindley (1997)
- *Basic Category Theory for Computer Scientists* by Pierce (1991)
  - More category theory: Jacobs (1999), Asperti and Longo (1991), Crole (1994)
- *Proofs and Types* by Girard, Lafont, and Taylor (1989)
- *Computation and Deduction* by Pfenning (2001)
- *Type Theory and Functional Programming* by Thompson (1991)
- *Constructive Foundations for Functional Languages* by Turner (1991)

[3]Many important advancements have been removed. This is a select choice based on personal views.

2. Mathematical Preliminaries

Common notation and mathematical facts required for the textbook.

2.1. Sets, Relations, and Functions

- We write exhaustive construction as $\{a, b, c, ...\}$, and the comprehension definition as $\{x \in S \mid ...\}$.
- The cardinality (size) of a set S is $|S|$. Its power set (the set of subsets of S) is $\mathcal{P}(S)$. $|\mathcal{P}(S)| = 2^{|S|}$
- The difference between two sets $S \setminus \mathcal{T} = \{s \in S \mid s \notin \mathcal{T}\}$.
- Cartesian product between two sets $S \times \mathcal{T} = \{s \in S, t \in \mathcal{T} \mid (s, t)\}$
- The empty set (set of 0 elements) is defined as $\varnothing$. The set of natural numbers $\mathbb{N} = \{0, 1, 2, ...\}$
- n-place *relation* on $(S_1, S_2, ..., S_n)$ is a set $R \subseteq S_1 \times S_2 \times ... \times S_n$. We say any tuple $s \in R$ is *related* by R.
- When $n = 1$ (one-place relation), it is called a *predicate* P on S. We often represent $s \in P$ as $P(s)$, i.e. $P(s) = s \in P$.
- When $n = 2$ (two-place relation), it is called a *binary relation over* S and $\mathcal{T}$. We often represent $(s, t) \in R$ as $s \ \mathcal{T} \ r$. When $S = \mathcal{U}$ and $\mathcal{T} = \mathcal{U}$ for some $\mathcal{U}$, we say R is a binary relation over $\mathcal{U}$.
- When $n \geq 3$, mixfix syntax is often used for relations with a sequence of symbols that represents the relation, e.g. $\Gamma \vdash s : T$ for (Γ, s, T) is in the relation (explained later in the book).
- The domain of a relation R on sets S and T is $\text{dom}(R) = \{s \in S \mid \exists t(s \ R \ t)\}$.
  The codomain is $\text{range}(R) = \{t \in S \mid \exists s(s \ \ R \ t)\}$.
- A relation R over S, T is called a *partial function* from S to T if

$$\forall t_1, t_2 \exists s \in S((s, t_1) \in R \land (s, t_2) \in R \to t_1 = t_2)$$

Alternatively, the contrapostive, which has its own meaning:

$$\forall (t_1, t_2) \in S^2 \exists s \in S(t_1 \neq t_2 \longrightarrow (s, t_1) \notin R \lor (s, t_2) \notin R)$$

- If $\text{dom}(R) = S$ for a partial function R, then R is instead called a *total function* (or just *function*).
- A partial function R is said to be *defined* on an argument $s \in S$ if $s \in \text{dom}(R)$ (often noted as $R(s) \downarrow$), and undefined if $s \notin \text{dom}(R)$ (or $R(s) \uparrow$).
- Note that "failure" is still defined, and is actually equivalent to being a function from S to $T \cup \{\text{fail}\}$, where failure returns the fail unit value. This is different from divergence, where there is no result at all (i.e. $f(x) \uparrow$).
- A predicate P is said to be *preserved* by a binary relation R if $\forall s, s'((s, s') \in R \land P(s) \to P(s'))$.

2.2. Ordered Sets

- A relation R on S is *reflexive* if $\forall s \in S((s, s) \in R)$. R is *symmetric* if $\forall s, t((s, t) \in R \to (t, s) \in R)$. R is *transitive* if $\forall s, t, u((s, t) \in R \land (t, u) \in R \to (s, u) \in R)$. R is *antisymmetric* if

  $$\forall s, t((s, t) \in R \land (t, s) \in R \to s = t)$$

  Alternatively, the contrapositive:

  $$\forall s, t(s \neq t \to ((s, t) \notin R \lor (t, s) \notin R))$$

- A reflexive and transitive relation R over a set S is called a *preorder* on S, often defined with $a \leq b$. A preorder that is also antisymmetric is called a *partial order*. A partial order is called a *total order* if $\forall s, t(s \leq t \oplus t \leq s)$.[4]

- Let $\leq$ be a partial order over S and $s, t \in S$. Then, $j \in S$ is a *join* (or *least upper bound*) if $s \leq j \land t \leq j$ and $\forall (k \in S)(s \leq k \land t \leq k \to j \leq k)$. Also, $m \in S$ is a *meet* (or *greatest lower bound*) if $m \leq s \land$

[4] I also suppose that a partial order R is a total order if $|R| = \frac{|S|(|S|+1)}{2}$

$m \leq t$ and $\forall(n \in S)(n \leq s \wedge n \leq t \to n \leq m)$. In a total order, these two definitions are represeted as $\max(s,t)$ and $\min(s,t)$, respectively.

- A reflexive, transitive, and symmetric relation R on a set S is called an *equivalence* (or *equivalence relation* on S). It can also be said that R *partitions* S.
- The *reflexive closure* of a binary relation R over S is the smallest (i.e. minimizing $|R'|$) reflexive relation R' such that $R \subseteq R'$. The *transitive closure* of R is the smallest (again, minimizing $|R'|$) transitive relation R' such that $R \subseteq R'$, often written as R^+. The combination is often written R^*.
- **Exercise**. Show that $R \cup \{(s,s) \mid s \in S\}$ is the reflexive closure of a binary relation R over S.
  This is the case as, by definition, the reflexive closure R' of R fulfills $\{(s,s) \mid s \in S\} \subseteq R$, so once we guarantee this condition is true by adding each of those elements, R' is correctly produced. Removing any element would fail to guarantee it is the transitive closure, and duplicates are not an issue as we are dealing with sets.
- **Exercise**. Given a constructive definition of R^+ as the union of every element in the recurrence relation as

$$R^+ = \bigcup_i R_i \quad \text{over} \quad \begin{aligned} R_0 &= R \\ R_n &= R_{n-1} \cup ((s,u) \mid \exists t((s,t) \in R \wedge (t,u) \in R)) \end{aligned}$$

  show that R^+ is really the transitive closure of R.
  Since the definition of transitivity indicates a relation is not transitive if

$$\exists(s,t,u \in S)((s,t) \in R \wedge (t,u) \in R \wedge (s,u) \notin R)$$

  it is clear that term R_n will ensure all pairs that could indicate the non-transitivity of R^{n-1} are added to R_n. As this process is done infinitely, if R is considered a directed graph over S, any two nodes with an n-length path between them must have a 1-length path between them by at latest R_n, fulfilling the transitivity of R^+.[5]
- Let $\leq$ be a preorder over S. A *decreasing chain* in $\leq$ is a sequence $s_1, s_2, ..., s_n$ such that $\forall i\left(s_{i_1} < s_i\right)$. In this case, S is said to be *well-founded* if there does not exist a decreasing chain of infinite length in $\leq$, or sometimes simply that S is well-founded (see the well-ordering_principle).

2.3. Sequences

- A *sequence* is an ordered list of elements, separared by comments. Commas are used both as `cons` for adding a single element to either end of a sequence as well as `append` for concatenating sequences. For example, if $a = 1,2,3$ and $b = 4,5,6$, then $0,a = 0,1,2,3$; $a,0 = 1,2,3,0$; and $b,a = 4,5,6,1,2,3$.[6]
- The sequence of numbers from 1 to n is written as $1..n$.
- The length of a sequence a is $|a|$.
- The empty sequence is either $\bullet$ or blank.
- Two sequences are (mutually) *permutations* of each other if they contain the same elements when discounting order.

2.4. Induction

Weak induction (also known as *ordinary induction*):

- For a predicate $P \subseteq \mathbb{N}$, if $P(0)$ and $\forall i(P(i) \to P(i+1))$, then $\forall n(P(n))$.

[5] It seems one could make a more accurate estimate of $\lceil \log_2(n) \rceil$ steps, but I am unwilling to attempt to prove this at the current moment. Also, I am aware this is far from a proof; I simply explain enough to convince myself.

[6] This does also mean that, under these operators, you cannot construct sequences of sequences.

Strong induction (also known as *complete induction*):
- For a predicate $P \subseteq \mathbb{N}$, if $\forall i(\forall j(j < i \to P(j)) \to P(i))$, then $\forall n(P(n))$.

Lexicographic order: order on pairs of natural numbers such that

$$(m, n) \leq (m', n') \longleftrightarrow ((m < m') \vee (m = m' \wedge n \leq n'))$$

Lexicographic induction:
- For a predicate $P \subseteq \mathbb{N}^2$, if $\forall(m, n, m', n')(((m', n') \leq (m, n) \to P(m', n')) \to P(m, n))$, then $\forall(m, n)P(m, n)$.
- Lexicographic induction is the basis for *nested induction* proofs.

Structural induction will be introduced in the future.

2.5. Background Reading

- Induction: Winskel (1993)
- Ordered sets: Davey and Priestley (1990)
- Basic set theory: Halmos (1987)

3. Untyped Arithmetic Expressions

We start with basic program semantics which we can be more clear about. We consider a simple language over numbers and booleans, which is basic but necessary before addressing other fundamental concepts.

3.1. Introduction

We consider a few basic operations: booleans, conditionals, 0, predecessor/successor, and `iszero`.

t ::=	*terms:*
`true`	*constant true*
`false`	*constant false*
`if t then t else t`	*conditional*
`0`	*constant zero*
`succ t`	*successor*
`pred t`	*predecessor*
`iszero t`	*zero test*

This resembles BNF from Aho, Sethi, and Ullman (1986). This syntax declares a set of *terms*, with t ranging over terms (any t in the grammar indicates that any term may be substituted in its place), and each line declaring a *form* of valid term. The terms on the right are comments.

t,[7] here, is a *metavariable*–a "variable" as it is a placeholder for a term, and "meta" because it is a variable in the *metalanguage* that is being used to describe the present *object language*.[8]

A *term* is an expression representing some sort of computation, while an *expression* is a general phrase, such as a term, type, or kind expression.

A *program* is a term built from the forms given by a grammar. Notably, they all must be based around `true`, `false`, or `0`, as those are the only forms that do not depend on other terms.[9]

An example program:

```
  if false then 0 else 1
▶ 1

  iszero (pred (succ 0))
▶ true
```

A few notes:

- The ▶ symbol is used to display the results of expressions, although it is omitted for trivial ones.
- Parentheses are used here for readability, although they are not in the grammar and do not matter for this language due to its unambiguous syntax.
- We use standard arabic numerals for numbers, represented with Peano arithmetic as nested applications of `succ` to `0`.
- Results are always booleans or numbers. These are called *values*. This means values, in this grammar, cannot include forms `if t then t else t`, `pred t`, and `iszero t`.
- Some permitted terms are nonsensical, like `succ true`. Disallowing terms such as these is the point of a type system.

[7] t, nearby letters like s, u, r, and variants such as t_1 and s' are also used as terms of the relevant object language.

[8] Also, the term *metatheory* refers to the collection of true statements about some logical system, so "metatheory of subtyping" refers to "the formal study of the properties of systems with subtyping".

[9] Disregarding infinitely recursive terms.

3.2. Syntax

The grammar in the previous section is a compact notation for the following inductive definition:

Definition: The set of *terms* is the smallest set $\mathcal{T}$ such that
1. $\{\texttt{true}, \texttt{false}, \texttt{0}\} \subseteq \mathcal{T}$
2. $t_1 \in \mathcal{T} \to \{\texttt{succ}\ t_1, \texttt{pred}\ t_1, \texttt{iszero}\ t_1\} \subseteq \mathcal{T}$
3. $\{t_1, t_2, t_3\} \subseteq \mathcal{T} \to \texttt{if}\ t_1\ \texttt{then}\ t_2\ \texttt{else}\ t_3 \in \mathcal{T}$.

This indicates a set of base terms, defines rules for constructing new terms, and indicates that only terms in this definition are in $\mathcal{T}$ (via "smallest"). Once again, this does not specify anything about parentheses; it defines a tree, not a set of strings. Parentheses are uses simply for clarifying or overriding precedence when converting expressions to strings.

It is also possible to define terms with inference rules:

$$\frac{}{\texttt{true} \in \mathcal{T}} \qquad \frac{}{\texttt{false} \in \mathcal{T}} \qquad \frac{}{\texttt{0} \in \mathcal{T}}$$

$$\frac{t_1 \in \mathcal{T}}{\texttt{succ}\ \mathcal{T}_1 \in \mathcal{T}} \qquad \frac{t_1 \in \mathcal{T}}{\texttt{pred}\ \mathcal{T}_1 \in \mathcal{T}} \qquad \frac{t_1 \in \mathcal{T}}{\texttt{iszero}\ \mathcal{T}_1 \in \mathcal{T}}$$

$$\frac{t_1 \in \mathcal{T} \qquad t_2 \in \mathcal{T} \qquad t_3 \in \mathcal{T}}{\texttt{if}\ t_1\ \texttt{then}\ t_2\ \texttt{else}\ t_3 \text{ in cal(T)}}$$

This is a similar procedure for *generating* the elements of $\mathcal{T}$; we start with known elements, and then introduce further production rules.

Premises are placed above line, with the conclusions that can be drawn from each premise included below the line. Rules with no premises (the top row) are called *axioms*. An *inference rule* refers to both axioms and *proper rules*. Inference rules are, in reality, *rule schemas*, since they can be instantiated on any valid *metavariables* to obtain a *concrete rule* that can be applied in a real case.

We can also define these inductively:

$$\begin{aligned} S_0 &= \varnothing \\ S_{i+1} &= \quad \{\text{true}, \text{false}, 0\} \\ &\quad \cup \{\text{succ}\ t_1, \text{pred}\ t_1, \text{iszero}\ t_1 \mid t_1 \in S_i\} \\ &\quad \cup \{\text{if}\ t_1\ \text{then}\ t_2\ \text{else}\ t_3 \mid t_1, t_2, t_3 \in S_i\} \end{aligned}$$

$$S = \bigcup_i S_1$$

Essentially, we define iterations of rule applications, and collect them all into S.

Exercise. How many elements does S_3 have?

This represents a recurrence relation of:

$r_0 = 0$

$r_{n+1} = 3 + 3r_n + (r_n)^3$

We sample up to r_3:

$r_0 = 0, r_1 = 3, r_2 = 39, r_3 = 59439$

Exercise. Show that the sets S_i are *cumulative*—that is, $\forall i(S_i \subseteq S_{i+1})$.

Instead of a proof, we provide the explanation that the definition for the next S_{i+1}, the iteration operation applies one more layer of indirection from the base value set, also adding the aforementioned set. Thus, it turns $\{0\}$ into $\{0+1\} \cup \{0\} = \{0, 1\}$, $\{0, 1\}$ into $\{0+1, 1+1\} \cup \{0\} = \{0, 1, 2\}$, et cetera.

Each set S_i, then, essentially contains the iterations equal to $\mathbb{Z}/i\mathbb{Z}$, which is a subset of $\mathbb{Z}/(i+1)Z$. It's possible to explain this in a much more sound manner, but that's too much work for this question.

The paper proceeds with a proof that $T = S$, which is not necessary here.

3.3. Induction on Terms

For any $t \in \mathcal{T}$, t must either:

- Be a constant: $t \in \{\text{true}, \text{false}, 0\}$
- Have the form succ t_1, pred t_1, or iszero t_1 for some *smaller* term t_1
- Have the form if t_1 then t_2 else t_3 for smaller terms t_1, t_2, t_3.

This can be used to give *inductive definitions* of functions over a set of terms, and to give *inductive proofs* over terms. For example, we define the number of nodes in the abstract syntax tree of a term t:

$$\begin{aligned}
\text{size}(\text{true}) &= 1 \\
\text{size}(\text{false}) &= 1 \\
\text{size}(0) &= 1 \\
\text{size}(\text{succ } t_1) &= \text{size}(t_1) + 1 \\
\text{size}(\text{pred } t_1) &= \text{size}(t_1) + 1 \\
\text{size}(\text{iszero } t_1) &= \text{size}(t_1) + 1 \\
\text{size}(\text{if } t_1 \text{ then } t_2 \text{ else } t_3) &= \text{size}(t_1) + \text{size}(t_2) + \text{size}(t_3) + 1
\end{aligned}$$

Taking `depth` to mean the depth of a term as a tree, we define some principles of induction on terms, supposing that $P : \mathcal{T} \to \text{boolean}$:

Over size: $\forall s(\forall r(\text{size}(r) < \text{size}(s) \land P(r)) \to P(s)) \to P(s)$

Over depth: $\forall s(\forall r(\text{depth}(r) < \text{depth}(s) \land P(r)) \to P(s)) \to P(s)$

Structural induction: $\forall s(\forall r(r \text{ is an immediate subterm of } s \land P(r)) \to P(s)) \to P(s)$

Notice that these examples, converted from the textbook, are partially strong induction (the first two examples) and partially weak induction (the last example).

Exercise. Prove the examples above. For brevity, only the last example will be proven.

We take $R(s)$ to equal the set of immediate subterms of s.

Proof: We aim to prove that, for any, $P : \mathcal{T} \to \text{boolean}$, structural induction holds for P over $\mathcal{T}$ such that, if $\forall s(\forall r \in R(s)(P(r))) \to P(s)$, then P holds for all s.

We first note that for terminal terms, there exist no subterms, so $\forall(r \in \text{subterms of s})P(r) = \forall\varnothing P(r)$, which is true.

For the inductive step–We make the inductive assumption that $\forall r \in R(P(r))$. Per modus ponens, we can use the assumption $\forall s(\forall r \in R(s)(P(r))) \to P(s)$ to prove that $P(s)$. Thus, the given $P(s)$ has been proven. Note that this is analogous to using $k-1$ to prove k instead of using k to prove $k+1$, as it was more convenient for this proof.

3.4. Semantic Styles

There are three main approaches to formalizing language semantics.

1. *Operational semantics*: behaviour is defined with an *abstract machine* formed from a *state* (which can be as simple as a term), defined with a *transition function*. The *meaning* of a term `t` is the final state the machine reaches when initialized with `t`. Providing multiple abstract machines is often useful for useful as a means of proving that language semantics are correct.
2. *Denotational semantics*: behaviour is abstracted from sequences of machine states to a more general idea of mathematical objects. Terms are mapped with an *interpretation function* into an element belonging to one of several *semantics domains*, known as *domain theory*. This omits the details of evaluation and makes it easier to reason about programs, such as through defining laws or identifying impossible behaviour.
3. *Axiomatic semantics*: the axiomatic laws themselves are the definition of the language instead of defining program behaviour and deriving laws from that.

Mathematically weak operational semantics were regarded as inferior to the other two, but proved more viable in the face of nondeterminism and concurrency (for denotational semantics), and procedures (for axiomatic semantics).

3.5. Evaluation

Let us now define the operational semantics of booleans.

$\mathbb{B}$ *(untyped)*

Syntax

`t` ::=		*terms:*
	`true`	*constant true*
	`false`	*constant false*
	`if t then t else t`	*conditional*
`v` ::=		*values:*
	`true`	*true value*
	`false`	*false value*

Evaluation $t \to t'$

$$\texttt{if true then } t_2 \texttt{ else } t_3 \to t_2 \quad \text{(E-IfTrue)}$$

$$\texttt{if false then } t_2 \texttt{ else } t_3 \to t_2 \quad \text{(E-IfFalse)}$$

$$\frac{t_1 \to t_1'}{\texttt{if } t_1 \texttt{ then } t_2 \texttt{ else } t_3 \to \texttt{if } t_1' \texttt{ then } t_2 \texttt{ else } t_3} \quad \text{(E-If)}$$

The column on the left represents the grammar defining two sets of expressions. t represents the possible terms, and v represents the values that can be emitted as a result of evaluation. v is used in the future to refer to the possible values.

The column on the right defines the *evaluation relation* on terms; t evaluates to t' in one step. Each rule in order:

- The first rule says that an `if true` can be literally evaluated to the `then` portion.
- The second rule says that an `if false` can be literally evaluated to the `then` portion.
- The third rule says that, if the guard t evaluates to t', then the whole conditional evaluats to `if` t_1' `then` t_2 `else` t_3. In essence, if the result of fully evaluating t_1 is t_1', then we can evalute this expression by evaluating `if` t_1' `then` t_2 `else` t_3.

This means that `true` and `false` can't be evaluated, and that in a statement such as

```
if true then (if false then false else false) else false
```

must have the outer `if` evaluated first; it cannot reach `if true then false else true` at any point. Referred to as an *evaluation strategy*, this corresponds with the common evaluation order of programming languages. The first two rules do the real work of evaluation ("*computation rules*") while the last rule simply "determines where work is to be done" (a "*congruence rule*").

Definition. An *instance* of an inference rule is where you reify each metavariable to an actual value, creating a concrete rule as explained prior.

Definition. A rule is *satisfied* by a relation if every instance of the rule has either a conclusion

Select References

References from *Types and Programming Languages* that seem interesting on an initial look.

- *Essentials of Programming Languages* by Friedman, Wand, and Haynes (2001)
- *Programming Language Pragmatics* by Scott (1999)